

EGRESS COMPLIANCE PLATFORM

Comprehensive Codebase Catalog, Function Architecture & Software Requirements Specification (SRS)

Document Identifier:	EGRESS-SPEC-SRS-2026-V1.0
Standard Compliance:	IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018 Systems Requirements
Jurisdictional Code:	UAE Fire and Life Safety Code of Practice (CDGH-OP-25), Ch. 3 Means of Egress
Target System:	EGRESS Automated CAD/PDF Building Review Platform (FastAPI + React + Supabase)
Author / Engineering:	DeepMind Advanced Agentic Systems & Senior FLS Architecture Team
Date of Publication:	August 30, 2026 (Revision 1.0 Production Release)
Classification:	Technical Architecture, Codebase Specification & Engineering Reference

EXECUTIVE SCOPE & PURPOSE

This master document fulfills two vital engineering needs for the EGRESS project: (1) A complete, exhaustive code catalog explaining what every file in the repository does, including class hierarchies, functions, parameters, return values, and mathematical algorithms; and (2) A formal Software Requirements Specification (SRS) conforming to IEEE 830 standards, specifying functional features, external interfaces, non-functional performance/security guarantees, database schemas, and the UAE Fire & Life Safety Code (FLSC 2018) compliance rules engine.

TABLE OF CONTENTS SUMMARY

Part I	System Architecture & Core Design Principles (Zero-Trust, Geometry Normalization, Dual-DB)
Part II	Repository File Catalog & Function-by-Function Breakdown (Every file, class & function)
Part III	Formal Software Requirements Specification (SRS - IEEE 830) (Sections 1 through 8)
Part IV	Data Dictionary, Database Schema & Mathematical Algorithms (NetworkX, Shapely, PyMuPDF)
Part V	UAE Fire & Life Safety Regulatory Citation Matrix (Table 3.8 to Table 3.22)

PART I: SYSTEM ARCHITECTURE & CORE PRINCIPLES

1.1 Executive System Overview

EGRESS is an enterprise-grade automated Fire & Life Safety (FLS) compliance platform designed for architects, fire safety engineers, municipal building authorities, and real-estate developers. It eliminates error-prone manual review of floor plans by parsing raw 2D CAD files (.dxf) and vector architectural PDFs (.pdf), reconstructing physical room polygons, egress doors, stairs, and corridors in true metric scale, and deterministically evaluating egress safety against the UAE Fire and Life Safety Code of Practice 2018 (Chapter 3: Means of Egress).

1.2 The Three Inviolable Architectural Principles

1. **Zero-Trust Occupant Load Determination:** EGRESS enforces an uncompromising zero-trust policy. Pre-written drawing text (such as 'Occ: 45' or stamped room capacities) is *never* trusted for legal safety evaluations because architects or tenants frequently underestimate capacities to evade exit requirements. Occupant load is calculated independently from physical room area (sq.m) divided by statutory occupant load factors (UAE FLSC Table 3.13) using strict ceiling rounding `math.ceil(area / factor)`.
2. **Real Physical Walkable Path Analysis:** Rather than using straight-line Euclidean 'as-the-crow-flies' distances, EGRESS constructs an orthogonal routing graph using NetworkX and Shapely. It maps actual travel from the deepest room centroid through the egress access doorway, down the central corridor spine, and directly to the nearest fire exit enclosure.
3. **Dual-Database Engine with Strict Citations:** The platform supports zero-config local development via SQLite with spatial JSON serialization, and seamlessly connects to cloud PostgreSQL (Supabase / AWS) via a custom cursor wrapper. Every flagged violation is bound to an exact clause ID, source table, and source page from the 1,348-page UAE FLS Code.

END-TO-END PIPELINE FLOW

Upload (.dxf / .pdf) -> Geometry Extraction (PyMuPDF vector rects / ezdxf LWPOLYLINE) -> Coordinate Normalization (0..100% SVG + true metric dimensions) -> 4-Tier Room Function Classifier -> Independent Occupant Load (Table 3.13) -> Walkable Path Graph (NetworkX) -> Rules Engine Evaluation (UAE FLSC 2018) -> Interactive React Visualizer + Multi-Floor Summary + CSV/PDF Audit Export

PART II: REPOSITORY FILE-BY-FILE BREAKDOWN & FUNCTION CATALOG

This section documents every file across all directories in the repository, detailing its functional purpose, internal classes, methods, arguments, return signatures, and integration relationships.

2.1 Backend Core Application Package (backend/app/)

File: `backend/app/main.py` (FastAPI Application & REST Routing)

Purpose: Central entry point for the REST API server. Configures CORS, lifecycle hooks, Pydantic schemas, database connection initialization, asynchronous file upload handling, multi-floor summary generation, high-resolution PDF page rasterization, and audit CSV streaming.

Function / Class	Parameters & Types	Description & Return Value
ProjectCreate (Pydantic Model)	<code>name: str, client_name: str, occupancy_type: str, sprinklered: bool</code>	Data transfer model validating project creation requests. Returns instantiated Pydantic model.
DrawingConfigUpdate (Pydantic Model)	<code>sprinklered: bool None, occupancy_type: str None, page_index: int None</code>	Validates real-time parameter changes that trigger automatic re-evaluation of code limits.
PageSelect (Pydantic Model)	<code>page_index: int</code>	Validates active floor page switching for multi-sheet drawing sets.
ViolationUpdate (Pydantic Model)	<code>status: Literal['confirmed', 'false_positive', 'resolved', 'open'], note: str None</code>	Validates reviewer audit overrides, statuses, and professional justification notes.
init_database()	<code>None</code>	Calls <code>db.init_db()</code> to build DDL schemas and checks if demo project 'project-al-noor' exists. If absent, executes <code>seed_demo()</code> .

feature(kind, coords, name, **props)	kind: str, coordinates: Any, name: str, **properties: Any	Constructs a valid GeoJSON Feature dictionary with specified geometry type, coordinates, and properties.
demo_elements()	None	Returns hardcoded GeoJSON fixtures for the fallback Al Noor Business Centre Level 06 office suite.
seed_demo(con)	con: sqlite3.Connection / Wrapper	Populates the database with demo project, drawing, elements, and 4 UAE code-cited violations.
serialize_element(row)	row: sqlite3.Row / DictCursor	Converts raw database element row into GeoJSON Feature, decoding JSON geometry and property strings.
serialize_violation(row)	row: sqlite3.Row / DictCursor	Serializes violation database record, unmarshaling spatial geometry into GeoJSON Point dictionary.
process_upload(drawing_id, page_index)	drawing_id: str, page_index: int None	Core processing orchestrator. Dispatches file to parse_dxf_file or parse_pdf_file, invokes calculate_walkable_distances, calculate_occupant_loads, and evaluate_fls_rules, then commits records.
compute_multi_floor_summary(drawing_id, con)	drawing_id: str, con: Connection	Iterates through all pages in a multi-page drawing PDF, computing rooms, occupant loads, travel distances, and violations per floor.
upload_drawing(...) [POST /projects/{id}/drawings]	project_id: str, file: UploadFile, occupancy_type: str, sprinklered: bool, scale: float	Receives uploaded file, writes binary to disk uploads/, initiates parsing for Page 0, returns drawing metadata & floor summary.
get_drawing_image(id, page) [GET /drawings/{id}/image]	drawing_id: str, page: int None	Uses PyMuPDF pixmap rendering (dpi=220) to generate crisp PNG raster representation of vector PDF page. Returns StreamingResponse.
select_drawing_page(...) [POST /drawings/{id}/page]	drawing_id: str, payload: PageSelect	Switches active viewing floor in multi-sheet drawing, re-executing extraction and returning updated floor metrics.
update_drawing_config(...) [PATCH /drawings/{id}/config]	drawing_id: str, payload: DrawingConfigUpdate	Updates sprinkler status or occupancy type and re-evaluates all code rules against modified thresholds.
export_summary(id) [GET /drawings/{id}/export]	drawing_id: str	Generates RFC 4180 CSV export of all findings, including clause references, measured vs limit values, reviewer notes, and status.

File: backend/app/db.py (Dual-Database Connection & Compatibility Engine)

Purpose: Manages persistent storage. Provides seamless dual-engine operation supporting zero-configuration local SQLite3 and cloud PostgreSQL (Supabase / Render). Translates parameter placeholders ('?' to '%s') and converts SQLite-specific 'INSERT OR REPLACE' into PostgreSQL 'INSERT ... ON CONFLICT DO UPDATE'.

Function / Class	Parameters & Types	Description & Return Value
get_raw_db_url()	None	Inspects environment variables (USE_LOCAL_SQLITE, DATABASE_URL, SUPABASE_DB_URL). Returns active database connection URL.
is_postgres()	None	Returns boolean True if active connection string begins with 'postgres://' or 'postgresql://'.
PostgresCursorWrapper (Class)	raw_cursor: psycopg2.cursor	Emulates sqlite3.Cursor over psycopg2. Rewrites '?' to '%s', intercepts 'INSERT OR REPLACE' queries to build ON CONFLICT clauses.
PostgresConnectionWrapper (Class)	raw_conn: psycopg2.connection	Emulates sqlite3.Connection. Provides cursor() with psycopg2.extras.DictCursor for dict-like column access.
get_db() (Context Manager)	None	Yields an active database connection with automatic transaction commit on exit and rollback on unhandled exception.
init_db()	None	Executes DDL schema creation for 5 tables: projects, drawings, extracted_elements, violations, code_clauses. Automatically calls load_code_clauses.
load_code_clauses(con)	con: Connection	Parses uae_fls_code_clauses_business_occupancy.json, populates statutory requirement values, source tables, and source pages. Returns clause count.

File: backend/app/dxf_parser.py (CAD DXF Geometry Extraction Engine)

Purpose: Parses Autodesk DXF CAD drawings using ezdxf. Inspects modelspace entities (LINE, LWPOLYLINE, INSERT, TEXT, MTEXT), identifies wall perimeters, room boundaries, doors, and exit stairs via layer categorization, computes true metric coordinates (detecting millimeters vs meters), and maps coordinates to normalized 0..100% SVG viewports.

Function / Class	Parameters & Types	Description & Return Value
DXFParseError (Exception)	msg: str	Custom exception raised when DXF file is unreadable, empty, or lacks valid 2D spatial geometry.
normalize_layer_name(layer)	layer: str	Strips whitespace and converts layer string to uppercase for case-insensitive matching.
is_wall_layer(layer)	layer: str	Returns True if layer name contains architectural wall keywords ('WALL', 'W_EXT', 'PARTITION', 'STRUCTURE', 'A-WALL').
is_room_layer(layer)	layer: str	Returns True if layer name signifies an enclosed boundary ('ROOM', 'SPACE', 'AREA', 'ZONE', 'A-AREA', 'OFFICE').
is_door_layer(layer)	layer: str	Returns True if entity layer signifies an egress doorway ('DOOR', 'A-DOOR', 'DR', 'OPENING').
is_exit_layer(layer)	layer: str	Returns True if layer signifies a protected exit stair enclosure ('EXIT', 'STAIR', 'EGRESS', 'FIRE_EXIT', 'S-01').
parse_dxf_file(path, scale)	file_path: str Path, drawing_scale: float = 100.0	Primary DXF parser. Extracts texts, lines, and polylines; calculates polygon areas using Shapely; associates room names; normalizes coordinates to 0..100% SVG viewport. Returns structured dictionary.

File: backend/app/pdf_parser.py (Architectural PDF Vector Parsing Engine)

Purpose: Extracts architectural geometry directly from vector PDF floor plans using PyMuPDF (fitz). Analyzes vector drawing commands ('re' for rectangles, 'l' for walls), extracts textual annotations and room function labels, detects multi-page floor sets, and normalizes bounding boxes into 0..100% viewport space.

Function / Class	Parameters & Types	Description & Return Value
PDFParseError (Exception)	msg: str	Raised when PDF document is corrupt, password-protected, or has 0 extractable pages.
get_pdf_pages_metadata(path)	file_path: str Path	Inspects all pages in an architectural PDF and extracts floor titles ('Ground Floor', 'Level 01') using keyword matching.
parse_pdf_file(path, page_idx)	file_path: str Path, page_index: int = 0	Universal PDF parser. Checks for benchmark layouts or dispatches to _parse_generic_vector_pdf for dynamic extraction.
_parse_generic_vector_pdf(...)	page: Any, raw_blocks: list, all_drawings: list	Universal vector path analyzer. Extracts vector rectangles, matches room text names inside bounding boxes, and detects exit doors.
_package_elements(...)	rooms_data, walls_data, doors_data, width_m, height_m	Packages extracted rooms, walls, doors, and exits into standardized GeoJSON element tuples with physical dimensions.

File: backend/app/occupant_load.py (4-Tier Zero-Trust Load Calculation)

Purpose: Implements the core Zero-Trust occupancy engine. Ignores pre-written occupant counts on floor plans and calculates occupant load per room using geometric area (m²) and Table 3.13 density factors. Applies a 4-tier classification hierarchy (Assembly → Mercantile → Storage/Service → Business Default) and enforces strict ceiling integer rounding.

Function / Class	Parameters & Types	Description & Return Value
calculate_occupant_loads(...)	parsed_data: dict, con: Connection, default_occupancy: str	Iterates through all extracted rooms, executes 4-tier classification algorithm, looks up Table 3.13 factor, computes math.ceil(area / factor), tags assumption flags, updates GeoJSON properties.

File: backend/app/path_analysis.py (Graph Pathfinding & Travel Distance Analysis)

Purpose: Constructs a topological escape path network using NetworkX and Shapely. Converts 0..100% normalized coordinates into true physical meters, connects room centroids to access doors, corridor spine nodes, and fire exits, and computes shortest egress travel distances using Dijkstra's algorithm while generating SVG orthogonal connection polylines.

Function / Class	Parameters & Types	Description & Return Value
calculate_walkable_distances(...)	parsed_data: dict	Constructs NetworkX Graph G. Adds nodes for exits, corridor spine, room centroids, and doorways. Computes shortest path length via nx.shortest_path. Generates orthogonal SVG path lines.
to_meters(svg_x, svg_y) (Helper)	svg_x: float, svg_y: float	Translates normalized 0..100% viewport coordinates to true physical meters using floor sheet width_m and height_m.

File: backend/app/rules_engine.py (UAE FLSC 2018 Rules Evaluation Engine)

Purpose: Deterministic rule verification module. Queries statutory clauses from the database and checks extracted geometry across 6 topics: Travel Distance (Table 3.16), Two Exit Doors Required by Area (Table 3.19), Single Exit Door Permission (Table 3.19), Minimum Number of Exits (Table 3.14), Corridor Clear Width (Table 3.8), and Exit Remoteness Separation (Table 3.15a).

Function / Class	Parameters & Types	Description & Return Value
evaluate_fls_rules(...)	parsed_data: dict, con: Connection, drawing_id: str, element_id_map: dict, is_sprinklered: bool, occupancy_type: str	Executes statutory compliance checks across all 6 core topics. Returns a list of violation dictionaries with severity, limits, measured values, and exact legal citations.

2.2 Backend Scripts, QA Test Suites & Generators (backend/)

File Path	Primary Responsibility & Functions
backend/seed_code_clauses.py	Database seeding utility. Contains run_seed() which executes init_db() and loads load_code_clauses() into SQLite or Supabase Postgres.
backend/scripts/verify_citations.py	CLI verification spot-checker. Contains find_source_pdf(), extract_page_snippet(), and verify_clause() to extract text from the official 1,348-page UAE FLS Code PDF.
backend/test_api.py	Integration test suite using FastAPI TestClient. Tests health checks, project listing, multipart file uploads, page navigation, violation querying, and CSV exports.
backend/test_complex_fls_fallacies.py	Senior QA fallacy stress-tester. Tests mathematical ceiling invariants, unscaled pixel fallacies, zero-trust label overwrites, corrupted DXF files, and concurrency isolation.
backend/test_coordinate_accuracy.py	Verifies precision alignment between raw PDF vector positions and 0..100% SVG coordinates, ensuring bounding boxes match actual drawing sheets without drift.
backend/test_dubai_regression.py	Automated regression test verifying that Level 01 typical office floor maintains exactly 158 occupants, 18.69m travel distance, and identical results between DXF and PDF paths.
backend/test_extract_all_pdfs.py	Batch extraction tester evaluating all PDF floor plans in the repository to guarantee parser stability and zero unhandled exceptions.
backend/validate_all_floors.py	End-to-end benchmark runner. Executes complete pipeline across all 6 storeys (Levels 00-05) of the Dubai Commercial Building test set, printing full compliance reports.
backend/generate_project_pdf.py	ReportLab PDF generator producing the Executive Project Overview & Technical Specification whitepaper.
backend/generate_room_classification_pdf.py	ReportLab PDF generator explaining the 4-tier classification algorithm and zero-trust occupancy principles.
backend/generate_non_technical_guide.py	ReportLab PDF generator creating the non-technical stakeholder accuracy explainer whitepaper.

2.3 Frontend Application Package (src/ & root)

File Path	Component / Functions	Description & Responsibilities
src/main.jsx (90 KB, 2,246 lines)	App, ReviewCanvas, toUiFinding, SVGFloorPlanViewer	Master React application. Contains state management for active project, drawing upload, multi-floor tab selector, interactive zoomable SVG drawing viewer with pan/zoom, dynamic violation marker pins, reviewer audit drawer, and CSV downloader.
src/EgressHome.jsx (27 KB, 629 lines)	EgressHome, SquigglyWave, handleFileChange, handleDrop	Modern marketing landing page & quick-upload portal. Features Dubai skyline hero imagery, drag-and-drop file upload zone, occupancy configuration selectors, and feature showcase tabs.
src/styles.css (48 KB)	Design tokens, CSS variables, dark/light theme classes	Core CSS stylesheet defining glassmorphism effects, Crimson/Navy brand palettes, SVG floorplan overlays, pulsing radar violation pins, and responsive flexbox layouts.
src/egress.css (39 KB)	Egress component styles & utility classes	Modular CSS rules for landing page hero cards, squiggly wave accents, animated progress bars, and modal overlays.
index.html	HTML5 document root	Loads web fonts (Plus Jakarta Sans, Inter, Outfit, JetBrains Mono) and binds root <div id='root'> to src/main.jsx.
package.json	NPM project configuration	Configures Vite build toolchain, React 18 dependencies, and Lucide-React vector icons.
render.yaml & vercel.json	Deployment Infrastructure configs	Render blueprint deploying FastAPI Uvicorn backend with automated database seeding; Vercel config for single-page React frontend.

PART III: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Structured in formal accordance with IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018 standards.

1. INTRODUCTION

1.1 Purpose: This document specifies the complete functional and non-functional requirements for the EGRESS Automated Fire & Life Safety (FLSC) Compliance Engine. It serves as the binding technical specification for software engineering, quality assurance verification, and municipal regulatory audits.

1.2 Scope: The software ingests architectural floor plans (.dxf and .pdf), extracts geometric envelopes and room boundaries, identifies egress access points, calculates true occupant densities under UAE Fire Code Chapter 3, models escape routes using orthogonal graph networks, evaluates statutory limits, renders an interactive visual review canvas, and exports certified audit documentation.

1.3 Definitions, Acronyms, and Abbreviations:

Term	Definition
FLS / FLSC	Fire and Life Safety / UAE Fire and Life Safety Code of Practice (2018 Edition, CDGH-OP-25).
Occupant Load	The total number of persons that may occupy a building or portion thereof at any one time (Table 3.13).
Travel Distance	The walking distance from the most remote point in a room along the natural path of travel to an exit enclosure (Table 3.16).
Zero-Trust Policy	Systematic refusal to rely on unverified drawing text annotations; occupant loads are derived purely from geometry.
Exit Remoteness	Minimum physical separation required between two fire stair doors (1/3 of floor diagonal if sprinklered, 1/2 if non-sprinklered).

2. OVERALL DESCRIPTION

2.1 Product Perspective: EGRESS functions as a cloud-native or on-premise SaaS solution. It interfaces with client CAD/BIM drafting environments through standard interchange formats (DXF/PDF) and provides a browser-based review workstation.

2.2 User Classes and Characteristics:

- *Architectural Designers:* Upload iterative plan revisions to identify code non-compliances prior to municipal submission.
- *FLS Engineers & Reviewers:* Review flagged violations, inspect path lines, adjust building parameters, and add audit override notes.
- *Civil Defense Authorities:* Inspect formal compliance exports, verification citations, and geometric integrity certificates.

3. SYSTEM FEATURES & FUNCTIONAL REQUIREMENTS

Each requirement is tagged with a unique identifier and priority rating (P1 = Mission-Critical, P2 = High, P3 = Medium).

Req ID	Feature Name	Priority	Functional Specification
FR-01	Multipart Drawing Ingestion	P1	The system shall accept multipart uploads of .dxf and .pdf architectural drawing files. Disallowed file extensions shall return HTTP 415.
FR-02	Geometric Boundary Extraction	P1	The system shall parse closed polygons, perimeter wall lines, egress access doors, and protected fire stair enclosures, converting them to standard GeoJSON.
FR-03	Physical Coordinate Normalization	P1	The system shall detect CAD units (millimeters vs meters), establish drawing bounding boxes, and normalize coordinates to 0..100% SVG viewports with 5% margins.
FR-04	Zero-Trust Occupant Load Calculation	P1	The system shall calculate occupant load independently per room as $\text{math.ceil}(\text{area_m2} / \text{factor})$ using UAE FLSC Table 3.13. Pre-written drawing numbers shall be ignored.
FR-05	4-Tier Semantic Room Classification	P1	The system shall classify room functions into: (1) Assembly (1.4-4.6 m2/p), (2) Mercantile (2.8 m2/p), (3) Storage/Service (27.9 m2/p), or (4) Business Default (9.3 m2/p).
FR-06	Walkable Graph Pathfinding	P1	The system shall build a NetworkX corridor graph to compute the shortest physical walking distance from each room centroid to the nearest fire exit.
FR-07	Deterministic Code Rule Evaluation	P1	The system shall evaluate travel distances (Table 3.16), exit counts (Table 3.14), corridor widths (Table 3.8), and exit remoteness (Table 3.15a) against statutory limits.
FR-08	Multi-Floor Set Navigation	P2	The system shall detect all floor plans in multi-page architectural PDFs, compute metrics per floor, and permit instantaneous floor switching via <code>POST /drawings/{id}/page</code> .
FR-09	Interactive SVG CAD Viewer	P2	The frontend shall render a pan/zoom SVG visualizer overlaying room polygons, walking escape paths, fire stairs, and pulsing radar violation markers.
FR-10	Reviewer Audit Overrides	P2	The system shall permit authorized reviewers to update violation status ('confirmed', 'false_positive', 'resolved') with persistent audit justification notes.
FR-11	Dynamic Parameter Re-evaluation	P2	When building parameters (sprinklered: true/false, occupancy_type) are updated, the system shall re-evaluate all rules without requiring re-upload.
FR-12	Audit CSV & Citation Export	P2	The system shall stream certified RFC 4180 CSV reports citing exact UAE FLSC clause IDs, measured values, limit thresholds, and reviewer notes.

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interface: Responsive Single Page Application (React 18 + Vite) styled with Crimson/Dark Navy palette, responsive sidebar drawers, multi-floor tab bars, zoom/pan SVG controls, and interactive violation inspection cards.

4.2 Software Interfaces / REST API:

- `POST /projects` — Creates project container.
- `POST /projects/{id}/drawings` — Multipart file upload and automatic parsing.
- `GET /drawings/{id}/elements` — Returns GeoJSON FeatureCollection of extracted rooms, walls, doors, exits.
- `GET /drawings/{id}/violations` — Returns code violations with spatial point coordinates and legal citations.
- `PATCH /drawings/{id}/config` — Updates sprinkler/occupancy configuration and triggers re-evaluation.
- `POST /drawings/{id}/page` — Switches active floor sheet in multi-page drawing.
- `PATCH /violations/{id}` — Records reviewer status overrides and justification notes.
- `GET /drawings/{id}/export` — Streams certified CSV compliance report.

5. NON-FUNCTIONAL REQUIREMENTS

- 5.1 Performance & Latency:** Single floor CAD/PDF files (<10 MB) shall be completely parsed, geometry-reconstructed, and evaluated within 2,500 milliseconds. Multi-page drawing sets (up to 10 storeys) shall generate a complete summary within 6,000 milliseconds.
- 5.2 Mathematical Accuracy & Reliability:** Occupant load calculations shall never deviate from $\text{math.ceil}(\text{area} / \text{factor})$. Travel distance calculations shall operate strictly on true metric scale, preventing unscaled coordinate errors.
- 5.3 Security & Zero-Trust Verification:** Uploaded files shall be sanitized and stored in isolated storage directories. Direct execution of unvalidated drawing commands is prohibited. Database queries shall utilize parameterized statements to prevent SQL injection.
- 5.4 Maintainability & Extensibility:** Regulatory rules shall reside in database tables rather than hardcoded logic, allowing instant updates when new editions of the building code are promulgated.

6. DATA DICTIONARY & DATABASE ARCHITECTURE

The system persists entities across 5 relational tables with cascading referential integrity.

Table Name	Columns & Types	Description & Foreign Keys
projects	id (TEXT PK), name (TEXT), client_name (TEXT), created_at (TEXT), occupancy_type (TEXT), sprinklered (INT)	Root project record. Defines default building occupancy and sprinkler protection.
drawings	id (TEXT PK), project_id (TEXT FK), file_url (TEXT), file_type (TEXT), occupancy_type (TEXT), scale (REAL), status (TEXT), created_at (TEXT), sprinklered (INT), page_index (INT), floor_name (TEXT)	Uploaded drawing sheet. References projects(id) ON DELETE CASCADE. Tracks active floor page index and processing status.
extracted_elements	id (TEXT PK), drawing_id (TEXT FK), type (TEXT), name (TEXT), geometry (TEXT JSON), properties (TEXT JSON)	Physical elements (room, wall, door, exit). References drawings(id) ON DELETE CASCADE. Stores GeoJSON geometry and properties.
violations	id (TEXT PK), drawing_id (TEXT FK), type (TEXT), related_element_id (TEXT), clause_ref (TEXT), measured_value (REAL), measured_unit (TEXT), limit_value (REAL), limit_unit (TEXT), severity (TEXT), status (TEXT), note (TEXT), geometry (TEXT JSON), title (TEXT), detail (TEXT)	Flagged code infractions. References drawings(id) ON DELETE CASCADE. Stores measured vs statutory limit values and audit status.
code_clauses	clause_id (TEXT PK), topic (TEXT), occupancy (TEXT), requirement_type (TEXT), value (REAL), unit (TEXT), condition (TEXT), note (TEXT), source_table (TEXT), source_page (INT)	Statutory rules table populated from UAE FLSC 2018 Chapter 3. Contains statutory thresholds, source tables, and source pages.

PART IV: UAE FIRE & LIFE SAFETY REGULATORY MATRIX

Direct statutory mapping of Chapter 3 (Means of Egress) clauses implemented within the rules engine:

Clause ID	Source Table	Source Page	Rule Topic & Requirement Value
UAE-FLS-3.13-BUS-REG	Table 3.13	Page 285	Occupant Load Factor: Business Regular Offices = 9.3 sq.m/person .
UAE-FLS-3.13-BUS-CONC	Table 3.13	Page 285	Occupant Load Factor: Concentrated Workstations = 4.6 sq.m/person .
UAE-FLS-3.13-ASSM-CONC	Table 3.13	Page 284	Occupant Load Factor: Concentrated Assembly (Auditorium/Hall) = 0.65 sq.m/person .
UAE-FLS-3.13-ASSM-LESS-CO NC	Table 3.13	Page 284	Occupant Load Factor: Meeting Rooms / Cafeterias / Pantries = 1.4 sq.m/person .
UAE-FLS-3.13-STOR-GEN	Table 3.13	Page 286	Occupant Load Factor: Storage / Mechanical / Plant Rooms = 27.9 sq.m/person .
UAE-FLS-3.14-LT500	Table 3.14	Page 287	Number of Exits: Floor occupant load 1 to 499 = Minimum 2 remote exits .
UAE-FLS-3.14-500-1000	Table 3.14	Page 287	Number of Exits: Floor occupant load 500 to 1,000 = Minimum 3 remote exits .
UAE-FLS-3.14-GT1000	Table 3.14	Page 287	Number of Exits: Floor occupant load > 1,000 = Minimum 4 remote exits .
UAE-FLS-3.15A-REMOTE-S	Table 3.15.a	Page 288	Exit Remoteness (Sprinklered): Exits separated by >= 1/3 of floor diagonal .
UAE-FLS-3.15A-REMOTE-NS	Table 3.15.a	Page 288	Exit Remoteness (Non-Sprinklered): Exits separated by >= 1/2 of floor diagonal .
UAE-FLS-3.16-BUS-TD-S	Table 3.16	Page 293	Travel Distance (Sprinklered Business): Maximum allowable travel distance = 91.0 meters .
UAE-FLS-3.16-BUS-TD-NS	Table 3.16	Page 293	Travel Distance (Non-Sprinklered Business): Maximum allowable travel distance = 61.0 meters .
UAE-FLS-3.19-BUS-ROOM-AR EA	Table 3.19	Page 304	Two Exit Doors by Area: Business rooms exceeding 280.0 sq.m require >= 2 remote exit doors.
UAE-FLS-3.19-BUS-SINGLE-DOOR	Table 3.19	Page 304	Single Exit Door Allowance: Permitted only if room occupant load < 100 persons .
UAE-FLS-3.8-CORRIDOR-WIDTH	Table 3.8	Page 276	Corridor Minimum Clear Width = 1,200 mm (capacity requirement = 5 mm/person).

PART V: VERIFICATION & TESTING METHODOLOGY

Quality assurance is enforced across five automated test vectors in the test suite:

- Boundary & Invariant Testing (test_complex_fls_fallacies.py):** Asserts that fractional calculations (e.g. 9.30001 m² / 9.3) strictly round up to 2 occupants per code; asserts that stairwells generate exactly 0 occupants regardless of floor area.
- Scale & Coordinate Accuracy (test_coordinate_accuracy.py):** Asserts that bounding boxes in PDF vectors match exactly with the SVG viewport (0..100%), eliminating coordinate drift.
- Automated Regression Benchmarking (test_dubai_regression.py):** Validates that the Dubai 5-floor benchmark suite maintains identical outputs across DXF and PDF parsing pipelines.
- Full Building Multi-Floor Runner (validate_all_floors.py):** Executes the complete pipeline across all storeys (Levels 00-05) of the Dubai Commercial Building set.
- End-to-End API Integration (test_api.py):** Tests FastAPI client endpoints, HTTP error status codes, and CSV streaming.

CONCLUSION & ENGINEERING SIGN-OFF

The EGRESS Automated Fire & Life Safety Compliance Platform represents a robust, deterministic, zero-trust engineering solution that bridges architectural CAD/PDF drafting with statutory life safety verification. All classes, functions, routes, and requirements documented herein are fully implemented, tested, and validated in the production codebase.